Metrics

chukmunnlee@nus.edu.sg

© Copyright National University of Singapore

What Are Metrics?

- Quantifiable measure of a system's performance and/or resource utilization
  - A unit of measurement associated with a data point
    - Eg. rate of request an application is receiving
  - Reflects the state of the application or infrastructure at a specific point in time
- Metrics can be used to track
  - Behaviour - how an application or infrastructure component is performing
    - Eg. P99 response latency
  - Health - operational status of an application or infrastructure component
    - Eg. CPU and memory utilization
  - Business impact - how the application stack achieves business objectives
    - Eg. how are the error rate affecting the daily sales target

© Copyright National University of Singapore

Metric Type - Counter

- A metric value that can only increase viz. monotonically increasing
- Use to keep track of the number of occurrences of a discrete event
  - Eg. number of request
- Counters can be used to identify spikes in the application
  - When the rate increases sharply

© Copyright National University of Singapore

Counter - Time Series Data

- Sequence of chronological numerical data points (counter) from the same data source
  - Tracked at a fixed increment of time

Image from https://prometheus.io/assets/docs/tutorial/rate_example.png

© Copyright National University of Singapore

Counter - Time Series Analysis

- Tracking counters over time may reveal trends, cyclical movements and seasonal variations
  - Trend - long-term directional movement (increase, decrease, stable) in the counter that persist over an extended period
  - Cyclical - fluctuations that occurs over irregular periods, no fix duration
  - Seasonal - regular, predictable fluctuations that repeat over fixed periods
  - Irregularities - unpredictable fluctuations

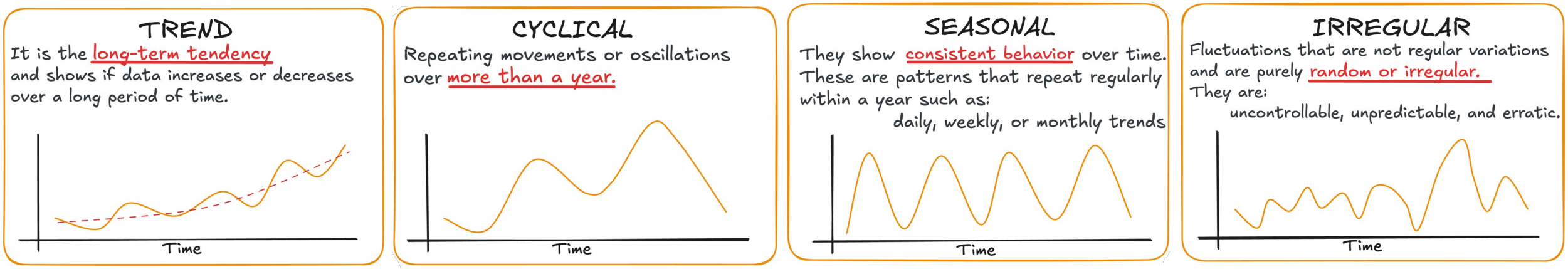


Image from https://www.kdnuggets.com/wp-content/uploads/Ferrer_Time_Series_2.png

© Copyright National University of Singapore

Metric Type - Gauge

- Measures the current value of a data point, can go up or down
  - Eg. memory and CPU utilization, number of active users
  - A snapshot of the instantaneous state
- Cannot use gauges to calculate rate

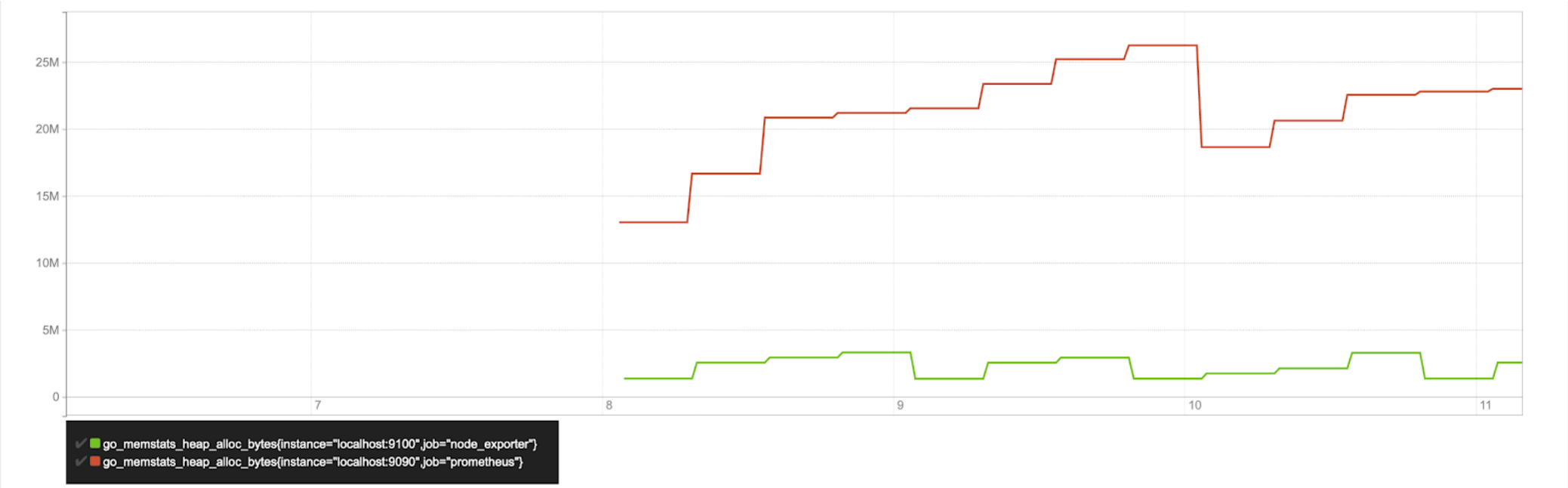
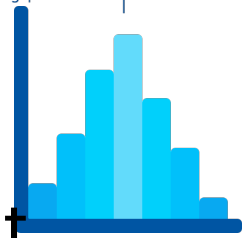


Image from https://prometheus.io/assets/docs/tutorial/gauge_example.png

© Copyright National University of Singapore

Metric Type - Histogram



- Measures the distribution and frequency of a specific data point
  - Eg. request processing time
  - Data points are groups into buckets/ranges
- Variation of histogram called summary (Prometheus)
  - For data points with unknown buckets

Bucket	Count
0 - 0.3	1
0 - 0.5	2
0 - 0.7	2
0 - 1	2
0 - 1.2	2
0 - +Inf	2

© Copyright National University of Singapore

Metric Record Structure

- OTLP metric record consist of the following major fields
  - Metric name - eg `http.server.request.duration`
  - Unit - measurement unit e.g. ms, sec
  - Data points
    - Attributes - key/value pairs used for filtering and grouping
      - Eg. `service.name=api,method=GET`
    - Timestamp - when the metric was taken
    - Value - value of the metric
    - Exemplars - co-relate with example traces
  - Type of metric
    - Sum/counter - monotonically increasing or decreasing value
    - Gauge - current value
    - Histogram - frequency and distribution
    - Exponential histogram - auto sized distribution

© Copyright National University of Singapore

Example - OTLP Metric Record

```
Metric #0
Descriptor:
     -> Name: http_request_total
     -> Description: Total number of requests
     -> Unit: int
     -> DataType: Sum
     -> IsMonotonic: true
     -> AggregationTemporality: Cumulative
NumberDataPoints #0
Data point attributes:
     -> http.request.method: Str(GET)
     -> url.path: Str(/search)
     -> service.name: Str(books)
     -> service.version: Str(1.0.0)
StartTimestamp: 2025-10-24 05:11:35.469 +0000 UTC
Timestamp: 2025-10-24 05:12:02.289 +0000 UTC
Value: 4.000000
```

© Copyright National University of Singapore

Choosing The Right Metrics

- Decide what metrics to measure
  - How to capture - determine the collection method and frequency eg. API request / 5 sec
  - How to query - storage and retrieval eg. Prometheus, InfluxDB, etc
- Decide if the metrics is actionable or exploratory metrics
  - Actionable metrics - metrics that leads to concrete actions, typically tied to business objectives, drives improvement
    - Non actionable metrics aka vanity metrics - does not have enough insight to make decision eg. number of product views without knowing if any of these view are converted to purchases
    - Eg. conversion rate, incident resolution time, up time
    - Eg. response latency to loss business opportunity, casual relationship between response time and conversion?
  - Exploratory metrics - help to investigate what is happening rather than measuring against a specific target
    - Discover trends, irregularities, cyclic, seasonal from multiple data source/angles
    - Eg. usage patterns, geographic distribution of activities, correlation between different user actions

© Copyright National University of Singapore

The Golden Signals

Latency

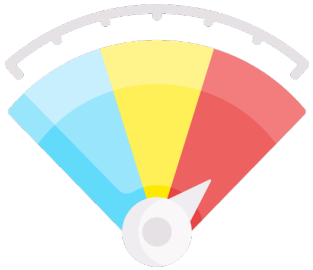
The time it takes to service a request, measuring how quickly a system responds. It's critical to track both successful and failed request latencies.

Errors

The rate of requests that fail, either explicitly (e.g., HTTP 500 errors) or implicitly (e.g., incorrect results), showing system reliability.

Traffic

The volume of demand on the system, such as requests per second, connections, or data throughput, indicating how much work the system is handling.



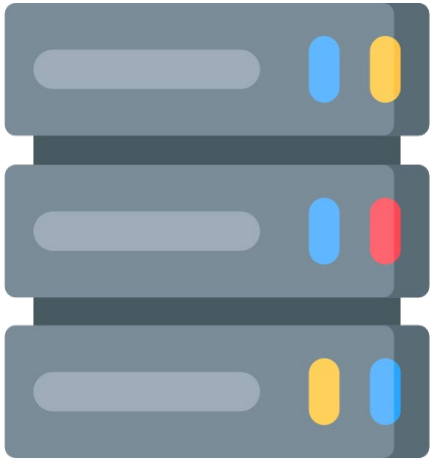
Saturation

The measure of resource utilization (e.g., CPU, memory, disk), indicating how "full" the system is and whether it's approaching capacity limits.

From https://sre.google/sre-book/monitoring-distributed-systems/#xref_monitoring_golden-signals

© Copyright National University of Singapore

Resource Focus



Utilization

Saturation

Error

- The resources that applications are deployed on and consume
- Eg. CPU, memory, disk, database connection pools, file descriptors
- USE
  - utilization - resource usage eg. CPU utilization = 70%
  - saturation - how much work is waiting for the resource eg. request waiting for db connection, active connection / max connections = 90%
  - errors - error count eg. number of time connection timeout

© Copyright National University of Singapore

User Experience

- Applications that handles user request and process business logic
- Eg. number of searches, number of active user sessions
- RED
  - rate - request per second eg. number of searches per second
  - error - failed request per second eg. number of failed checkout
  - duration - time to complete a request eg. number of P95 > 2sec

Request
Error
Duration

© Copyright National University of Singapore

Other Metrics

- Business metrics eg. number of shopping carts, average time spend in the application
- Security and compliance eg. rate of failed logins

© Copyright National University of Singapore

KPI - Reliability Engineering Metrics

SLA —Promise→

Service Level Agreement

Agreement between client and user

Eg. service uptime must be 99.9% per month

SLO —Objective→

Service Level Objectives

Objectives a service provider must achieve to meet the agreement

Eg. service uptime must be 99.95% per month

SLI —Measurement→

Service Level Indicators

Quantitative measure of the performance

Eg. measure the actual uptime

© Copyright National University of Singapore

Error Budget

- Acceptable amount of failure that a service can tolerate
  - while still meeting performance target
  - Eg. SLO is 99.99% uptime per month, then error budget is approximately 4.38 minutes
- Knowing the error budget provides an operational framework
  - Low error budget, focus on service reliability
  - High error budget, focus on new features/innovations
  - Balances reliability and innovation, can be strategically managed and allocated

© Copyright National University of Singapore

Capturing Metrics

© Copyright National University of Singapore

OpenTelemetry Metrics SDK

- The OpenTelemetry SDK powers the generation, aggregation and exporting metrics from an application
  - The metrics can be shipped to Prometheus directly or via OTel gateway
- Types of metric instruments
  - Counter - value that is monotonically increasing
  - UpDownCounter - like a gauge
  - Histogram - records distribution of measurements
- Two ways metrics are collection
  - Synchronous instrumentation - take a measurement at the time an event occurs, like a push
  - Asynchronous instrumentation - take a measurement only when the SDK collects it, like a pull
    - Via a callback function

© Copyright National University of Singapore

Metrics Collection Strategies

- OpenTelemetry can collect metrics the following ways
  - Application pushing metrics to OTel
  - OTel pulling (scraping) metrics from the application
- OpenTelemetry SDK supports both modes
  - Push with OTLP metrics exporter
  - ~~Push~~ Pull with Prometheus provider
- Prometheus uses pull-based model, viz. scrapes the application exposed metrics endpoint
  - Use the Prometheus provider if you want Prometheus to scrape your application

© Copyright National University of Singapore

OTel Metric SDK - Push

```
const { OTLPMetricExporter } = require('@opentelemetry/exporter-metrics-otlp-grpc')
const { PeriodicExportingMetricReader } = require('@opentelemetry/sdk-metrics')

const otlpMetricsExporter = new OTLPMetricExporter({
  url: 'http://otlp-gateway:4317'
})

const metricReader = new PeriodicExportingMetricReader({
  exporter: otlpMetricExporter,
  exportIntervalMillis: 15 * 1000 // Every 15 seconds
})

const sdk = new NodeSDK({
  metricReaders: [ metricReader ],
  ...
})
```

Create an instance of **`OTLPMetricExporter`** specifying the gRPC port of the OTLP gateway

Create an instance of **`PeriodicExportingMetricReader`** to forward metrics to the exporter at regular interval

Configure the OTel SDK to use **`metricReaders`** to read and export metrics

© Copyright National University of Singapore

OTel Metric SDK - Pull

```
const { PrometheusExporter } = require('@opentelemetry/exporter-prometheus')

const promExporter = new PrometheusExporter({
  port: 8000,
  endpoint: '/metrics'
})

const sdk = new NodeSDK({
  metricReaders: [ promExporter ],
  ...
})
```

Create an instance of **`PrometheusExporter`**

Expose the metrics on a separate port 8000 at **`/metrics`**. **`/metrics`** is the default

Configure the OTel SDK to use **`promExporter`** to allow metrics to be scraped

© Copyright National University of Singapore

Example - Counter Metrics

```
const { metrics } = require('@opentelemetry/api'}

const meter = metrics.getMeter(appName, appVersion)

const httpRequestTotal = meter.createCounter(
  'http_request_total', {
    description: 'Total number of requests',
    unit: 'int'
  }
)

httpRequestTotal.add(1, {
  [ ATTR_HTTP_REQUEST_METHOD ]: 'POST',
  [ ATTR_URL_PATH ]: '/checkout',
  [ ATTR_HTTP_STATUS_CODE ]: 201
})
```

Get the global OTel SDK metrics

Create a counter: name, description and the unit

Metrics name

Increment the metrics by 1

One or more attributes associated with the metric

© Copyright National University of Singapore

Example - Histogram Metrics

Predefined list of increasing numeric boundaries

[0, 5, 10, 25, 50, 75, 100, 250, 500, 750, 1000, 2500, 5000, 7500, 10000]

```
const httpRequestDuration = meter.createHistogram(
  'http_request_duration', {
    description: 'Duration of a request in milliseconds',
    unit: 'ms'
  }
)
```

Record the duration

```
httpRequestDuration.record(Date.now() - startTime,
{
  [ ATTR_HTTP_REQUEST_METHOD ]: 'POST',
  [ ATTR_URL_PATH ]: '/checkout',
  [ ATTR_HTTP_STATUS_CODE ]: 201
})
```

© Copyright National University of Singapore

Example - Gauge Metrics

Observable method only records the metric value when the OTel SDK requires it

```
const httpRequestInflight = meter.createObservableUpDownCounter(

  'db_pool_pending_requests', {
    description: 'Number of pending connection request',
    unit: 'int'
  },

  (updownCounter) => {
    updownCounter.observe(pool._acquiringConnections.length,{
      [ ATTR_DB_name ]: 'books',
    })
  }
)
```

Callback function used to record the observation when OTel SDK initiates the metric collection

Assume the following returns the number of outstanding waiting connection requests

© Copyright National University of Singapore

Naming Metrics - Best Practices

- Convention is to use snake case

  `snake_case`

- Follow the structure of **`namespace_subsystem_name_unit`**
  - Include unit eg. seconds, bytes, meters to the metric name

    `http_server_requests_duration_ms`

  - Counter metrics should end with total as its unit

    `http_server_requests_total`

- Keep label names consistent across metrics
- Avoid putting label values in the metric name

  `http_server_get_requests_total` 👎

  `http_server_requests_total{method="GET"}` 👍

© Copyright National University of Singapore

Avoid High Cardinality Metrics

- Avoid creating metrics with high cardinality labels eg emails, IP addresses, numbers

```
http_server_uploads_bytes{file_size=2779718}
http_server_request_total{method="GET", path="/api/sku/abc123"}
```

- Bucket the labels if the value is numeric eg. small, medium, large

```
http_server_uploads_bytes{file_size="small"}
```

- Cap the number of distinct values, select up front what are the critical values to be included
  - Eg. **ECONNREFUSED**, **ER_ACCESS_DENIED_ERROR**, **ER_BAD_DB_ERROR**, **OTHERS**

```
db_pool_status{code="ECONNREFUSED"}
```

© Copyright National University of Singapore

Avoid High Cardinality Metrics

- Use sampling when you absolutely require high-cardinality metrics
  - Eg. Only sample 20% of the metrics
- Correlate with other signals other signals
  - Use exemplar to correlate metrics to traces
- Use OTel SDK's client side aggregation to reduce metric volume before sending to backend viz. metrices are pre-aggregated on the application
  - See https://opentelemetry.io/docs/languages/js/instrumentation/#configure-metric-views
  - Types of client-side aggregation
    - Temporal aggregation - combine measurements over a time window
      - Eg. number of errors
    - Dimensional aggregation - drop or combine labels
      - Eg. drop high dimensionality
    - Statistical aggregation
      - Eg. average, median, P99

© Copyright National University of Singapore

Workshop 3
Task 1

© Copyright National University of Singapore

Metrics Architecture

© Copyright National University of Singapore

Prometheus

- An open source time-series database and a monitoring system
  - Started by SoundCloud in 2012
- Features
  - Multi dimensional data model identified by metric name and key/value pairs
  - Include a query language PromQL for querying stored metrices
  - Pre-dominantly a pull based collection with push gateways for ephemeral targets eg. batch jobs
  - Supports service discovery for dynamic environments like Kubernetes and container runtime (Docker)
  - Built in alerting
- Has storage for metrics, but not suitable for high volume, multi-tenanted data
  - Use with Thanos, Mimir, InfluxDB TSDB

© Copyright National University of Singapore

Prometheus Architecture

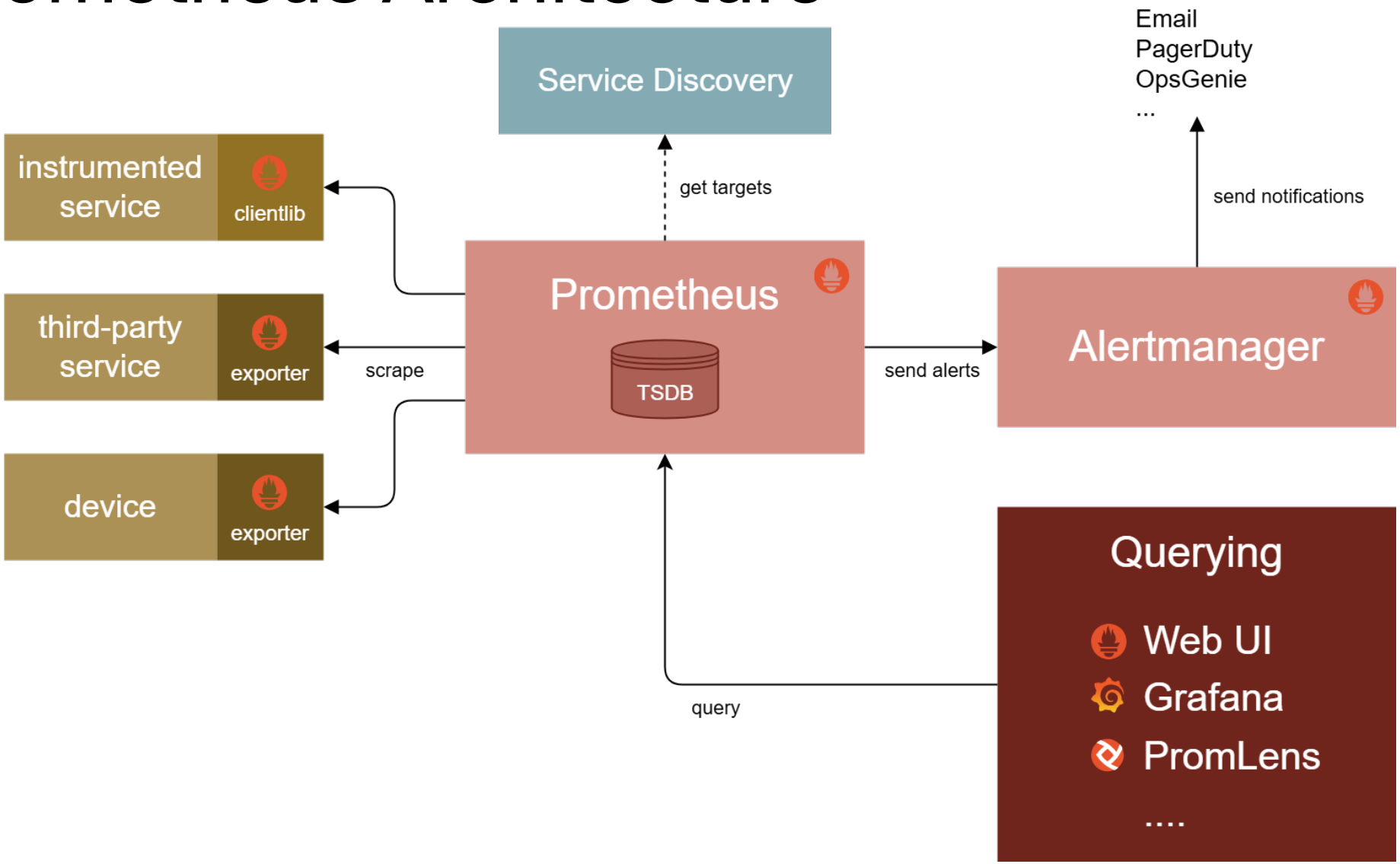


Image from https://training.promlabs.com/training/introduction-to-prometheus/prometheus-an-overview/system-architecture/

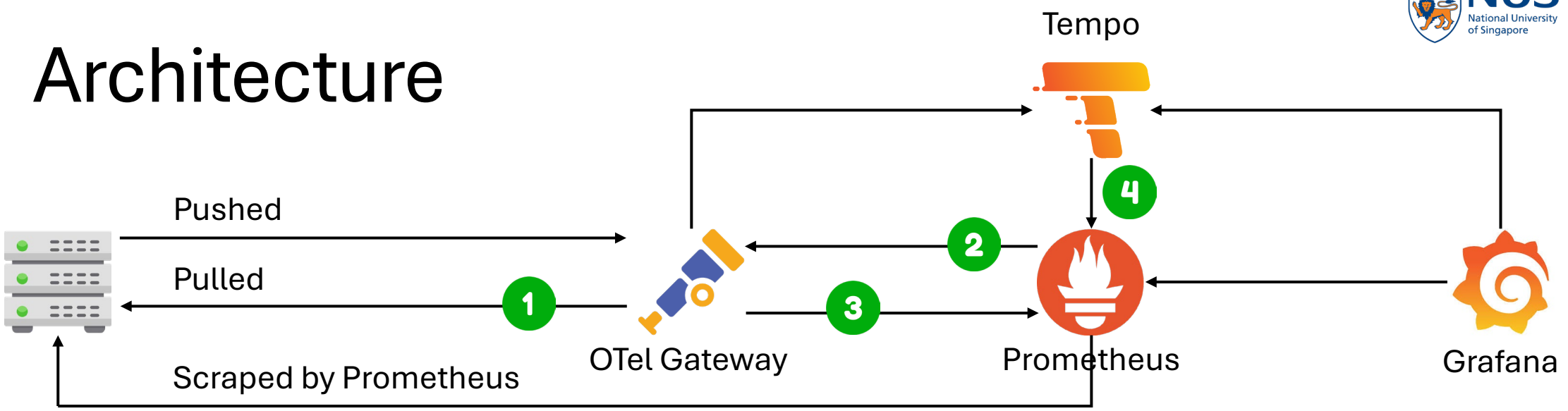
© Copyright National University of Singapore

Prometheus Architecture

- Prometheus can ingest metrics from
  - Static targets - scrape targets with static endpoints
    - Eg. IP or DNS name of a server
  - Dynamically discovered through service discovery - for dynamic environments like Kubernetes, cloud providers, DNS
    - Eg. configure a Kubernetes cluster as a target, Prometheus will dynamically discover pods and other metrics as they are created and destroyed
- Exporters are components that exports metrics extracted from other systems, configured as a static target
  - Eg. MySQL exporter, node exporter
  - See https://prometheus.io/docs/instrumenting/exporters/
- Alert manager sends alerts whenever thresholds are breached

© Copyright National University of Singapore

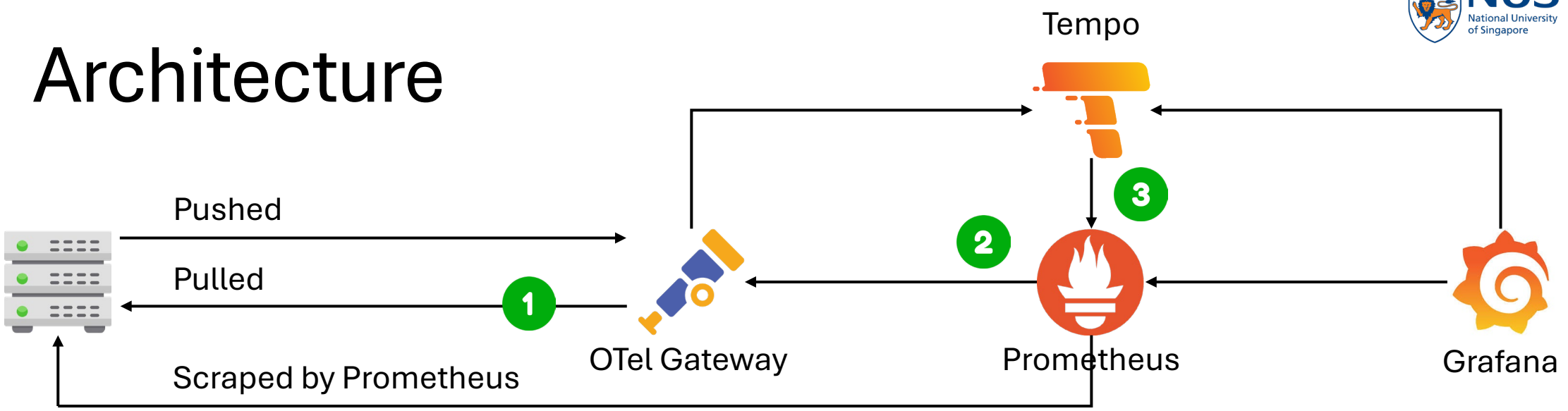
Architecture



- Application are either push to or pulled from the OTel gateway 1
  - Legacy applications written with Prometheus SDK (prom-client) may required metrics to be scraped
  - Exporting metrics directly to Prometheus
- Prometheus scrapes OTel gateway for metrics 2
- OTel gateway pushes metrics to Prometheus with remote write feature 3
- Tempo can use Prometheus store trace data for generating graphs 4
  - Traces information are written to TSDB like Prometheus and is used to calculate RED metrics

© Copyright National University of Singapore

Architecture



- Application are either push to or pulled from the OTel gateway 1
  - Legacy applications written with Prometheus SDK (prom-client) may required metrics to be scraped
  - Exporting metrics directly to Prometheus
- Prometheus scrapes OTel gateway for metrics 2
- Tempo can use Prometheus store trace data for generating graphs 3
  - Traces information are written to TSDB like Prometheus and is used to calculate RED metrics

© Copyright National University of Singapore

OTel Collector Gateway Configuration

Applications pushes their metrices to OTel gateway thru endpoint

OTel pulls metrics from these endpoint

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

  prometheus:
    config:
      scrape_configs:
      - job_name: node-app
        scrape_interval: 15s
        static_configs:
        - targets: [ 'myapp:8000' ]
```

© Copyright National University of Singapore

OTel Collector Gateway Configuration

```yaml
receivers:
  otlp:
    ...
  prometheus:
    ...

processors:
  batch:

exporters:
  prometheus:
    endpoint: 0.0.0.0:9090
    send_timestamps: true
    translation_strategy: UnderscoreEscapingWithSuffixes
```

Prometheus exporter exposes an endpoint for Prometheus to scrape

Convert metrics name to be compatible with Prometheus

```yaml
service:
  pipelines:
    metrics:
      receivers: [ otlp, prometheus]
      processors: [ batch ]
      exporters: [ prometheus ]
```

© Copyright National University of Singapore

Prometheus Configuration

```yaml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
- eng-dept.yaml

scrape_configs:
- job_name: otel-gateway
  tls_config:
    insecure_skip_verify: true
  static_configs:
  - targets:
    - otel-gateway:9090
```

Period to perform scrapes and to evaluate the rule files

One or more rule files. PromQL expressions are precomputed to improve query performance.

Logical name of scrape group

Group of logical scrape targets
Note that this is similar to OTel's Prometheus receiver

The OTel gateway Prometheus exporter endpoint

```
prometheus --config.file=prometheus.yaml
```

© Copyright National University of Singapore

Scrape Configs

- Prometheus supports many different types of scrape targets
  - Eg. static, Kubernetes, Docker, DigitalOcean, etc
  - See https://prometheus.io/docs/prometheus/latest/configuration/configuration/#scrape_config
- Include specific labels to scrape configs for easy metrics identification and categorization
  - Eg used in filtering

```
scrape_configs:
- job_name: prod_app

  static_configs:
    targets:
    - mysql-exporter.prod:9104

  kubernetes_sd_configs:
  - api_server: https://kubernetes.default
    role: pod
    namespaces:
      names: [ prod-ns ]

  labels:
    datacenter: east01
```

Labels will be included in the metrics scraped from this job group

© Copyright National University of Singapore

Workshop 3
Task 2

© Copyright National University of Singapore

PromQL

© Copyright National University of Singapore

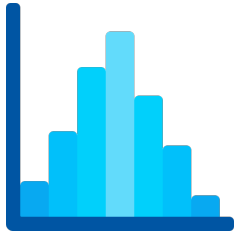
Type of Prometheus Metrics

Counters track values that can only increase

Eg. number of request

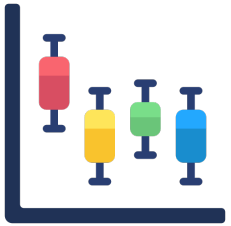
Gauges track values that can increase or decrease

Eg. temperature



Histograms track cumulative bucketed counts of observations

Eg. request duration



Quantiles reveal how observations are spread out

Eg. request latency quantiles

© Copyright National University of Singapore

Prometheus Metric Data Model

- A metric consist of the following
  - Name - identify what is being measured
  - Labels - key/value pairs that provide context to the metric
  - Timestamp - time when the measurement was taken
  - Type - describes how the metric is updated and aggregate
    - Counter, gauge, histogram, summary
- Counters and gauges include metric name and a set of associated labels

Example of a Prometheus exposition format

```
# HELP promhttp_metric_handler_requests_total Total number of scrapes by HTTP status code.
# TYPE promhttp_metric_handler_requests_total counter
promhttp_metric_handler_requests_total{code="200"} 4
promhttp_metric_handler_requests_total{code="500"} 0
promhttp_metric_handler_requests_total{code="503"} 0
```

Metric type

Name

One or more labels

Value

© Copyright National University of Singapore

Prometheus Metric Data Model - Histogram

- Histogram and summary are used to measure distribution of observed values eg. response latency
  - Histogram - track the distribution of observed values into a set of ranged buckets
  - Summary - track pre-computed quantiles, not aggregatable
- Histogram has the metric names appended with the following suffixes
  - **`_bucket{le="0.5", ... }`** - one bucket per metric where `le` label (less than or equals to) representing the upper bounds of the bucket
    - Eg. **`http_request_duration_ms_bucket{le="1000",...} 10`** - 10 request that took less than or equals to 1000ms
    - Special **`le`** value **`+Inf`**, this is the total number of observations
  - **`_count`** - total number of observations, same value as **`le="+Inf"`**
  - **`_sum`** - sum of all observed values

© Copyright National University of Singapore

Prometheus Metric Data Model - Summary

- Summary tracks snapshot of values in a time window
  - Eg P95 of checkout duration
- Calculation is performed on the application by the client SDK
  - Histograms are computed on the server by Prometheus
- Summary metrics has the following structure
  - quantile label - one metrics for each pre-defined quantile
    - Eg. `http_request_duration_ms{quantile=".95",...} 1300` - the 95 quantile/percentile is 1300ms
  - `_count`, `_sum` - same as histogram

© Copyright National University of Singapore

Example - Histogram and Summary

Histogram

```
http_request_duration_ms_bucket{method="GET",endpoint="/api/users",status="200",le="10"} 142
http_request_duration_ms_bucket{method="GET",endpoint="/api/users",status="200",le="25"} 289
http_request_duration_ms_bucket{method="GET",endpoint="/api/users",status="200",le="50"} 412
http_request_duration_ms_bucket{method="GET",endpoint="/api/users",status="200",le="100"} 587
...
http_request_duration_ms_bucket{method="GET",endpoint="/api/users",status="200",le="10000"} 858
http_request_duration_ms_bucket{method="GET",endpoint="/api/users",status="200",le="+Inf"} 858
http_request_duration_ms_sum{method="GET",endpoint="/api/users",status="200"} 98234.5
http_request_duration_ms_count{method="GET",endpoint="/api/users",status="200"} 858`
```

Summary

```
http_request_duration_ms{method="GET",endpoint="/api/users",status="200",quantile="0.5"} 45.2
http_request_duration_ms{method="GET",endpoint="/api/users",status="200",quantile="0.9"} 178.4
http_request_duration_ms{method="GET",endpoint="/api/users",status="200",quantile="0.95"} 287.6
http_request_duration_ms{method="GET",endpoint="/api/users",status="200",quantile="0.99"} 823.5
http_request_duration_ms{method="GET",endpoint="/api/users",status="200",quantile="0.999"} 1456.7
http_request_duration_ms_sum{method="GET",endpoint="/api/users",status="200"} 98234.5
http_request_duration_ms_count{method="GET",endpoint="/api/users",status="200"} 85
```

© Copyright National University of Singapore

Key Differences Between Histogram and Summary

Histogram	Summary
Fixed bucket boundaries (**`le`** labels) that are cumulative	Pre-computed quantiles (**`quantile`** label)
Each bucket contains count of all observations within the **`le`** boundary	Exact quantiles for the time window when calculated
Aggregatable	Non aggregatable, quantiles of quantiles are meaningless
Can approximate quantiles with **`histogram_quantile()`** function	
Both have **`_sum`** and **`_count`** metrics. Meaning of these are identical	

© Copyright National University of Singapore

PromQL Basics

- Language for querying Prometheus (time series database)
  - Query only, writes are performed on **`/api/v1/write`** endpoint
  - Performs filtering, aggregation and arithmetic operations on the metrics
- Basic PromQL syntax
  - Metric selector - returns all time series with the metric name

    `http_request_total`
  - Label selector - returns metrics that matches the labels

    `{ method =~ "GET|POST", uri_path = "/product/abc" }`
  - Metric and label selector - return the metric that matches the labels

```
http_request_total { method =~ "GET|POST", uri_path = "/product/abc" }
```

© Copyright National University of Singapore

PromQL Basics

- Basic PromQL syntax
  - Offset selector - offset the time from the current time by the specified duration

```
http_request_total offset 30m
http_request_total - (http_request_total offset 30m)
```

  - Range selector - returns all matching metrics from the specified duration, used with aggregation function

```
http_request_total { method =~ "GET|POST", uri_path = "/product/abc" }[5m]
```

- Label matching operators
  - **=** select labels that are equal to the provided string literal
    - == used for value comparison in expression
  - **!=** not equals to the provided string literal
  - **=~** select labels that matches the provided regex
  - **!~** does not match the provided regex

© Copyright National University of Singapore

Instance Vector

- Single metric value for each matching time series at that point in time
  - Return by queries without the range selector

```
>_ http_request_int_total
```

Execute

Table | Graph | Explain

Metric

Single value

Evaluation time

Load time: 61ms Result series: 4

http_request_int_total{**exported_job**="books", **http_request_method**="GET", **instance**="otel-gateway:9090", **job**="otel-collector", **otel_scope_name**="books", **otel_scope_version**="1.0.0", **url_path**="/book/:isbn13"}	12
http_request_int_total{**exported_job**="books", **http_request_method**="GET", **instance**="otel-gateway:9090", **job**="otel-collector", **otel_scope_name**="books", **otel_scope_version**="1.0.0", **url_path**="/books"}	19
http_request_int_total{**exported_job**="books", **http_request_method**="GET", **instance**="otel-gateway:9090", **job**="otel-collector", **otel_scope_name**="books", **otel_scope_version**="1.0.0", **url_path**="/search"}	17
http_request_int_total{**exported_job**="books", **http_request_method**="POST", **instance**="otel-gateway:9090", **job**="otel-collector", **otel_scope_name**="books", **otel_scope_version**="1.0.0", **url_path**="/book"}	22

© Copyright National University of Singapore

Example of Instance Vectors

- Use for creating graphs
  - Eg. service status in prod environment

```
up{ env = "prod" }
```

- Mathematical operations
  - Eg. total number of request over the last 30 minutes

```
http_request_total - (http_request_total offset 30m)
```

- Derived metrics
  - Eg. memory utilization

```
node_memory_Active_bytes / node_memory_MemTotal_bytes
```

- Filtering
  - Eg. resident memory is greater than 12GB

```
node_memory_Active_bytes > 12884901888
```

© Copyright National University of Singapore

Range Vector

- Contains multiple values for a single metric over a time range; produced by queries with range selector

© Copyright National University of Singapore

Build-in Metrics

- Prometheus automatically generates metrics for every scrape action it performs on a target
  - **`up`** - a gauge that indicates the health of a scrape targe
    - 1 means the target is healthy and was successfully scraped, 0 means the target was not successfully scraped

```
up{env="prod"} == 1
```

  - **`scrape_duration_seconds`** - a gauge that shows how long the most recent scrape took

```
scrape_duration_seconds{env="prod"} > 2
```

© Copyright National University of Singapore

Metric Aggregation - Cross Sectional

- Aggregate metrics by combining multiple time series across their labels
  - Operates on an instance vector
- Clauses which controls the grouping dimension
  - **`by`** - keeps only the listed labels from the result
  - **`without`** - removes the listed labels from the result
  - Only timeseries with the same label values can be combined
    - Use **`by`** and **`without`** keywords to intentionally ignore specific labels
- Common aggregation functions
  - **`sum`**, **`avg`**, **`min`**, **`max`**, **`count`** , **`stddev`** , **`stdvar`** , **`topk`** , **`bottomk`**
  - See https://prometheus.io/docs/prometheus/latest/querying/functions/

© Copyright National University of Singapore

Query Evaluation

```
http_request_int_total{service_name="api", status_code=200} 3
http_request_int_total{service_name="db", status_code=503} 8
http_request_int_total{service_name="api", status_code=401} 5
http_request_int_total{service_name="api", status_code=500} 3
```

```
sum by (service_name)
       (http_request_int_total{status_code=~"[45]..})
```

Sum all status code >= 400 by their service name

1 Filter the metrics

```
http_request_int_total{status_code=~"[45]..}
```

```
http_request_int_total{service_name="db", status_code=503} 8
http_request_int_total{service_name="api", status_code=401} 5
http_request_int_total{service_name="api", status_code=500} 3
```

© Copyright National University of Singapore

Query Evaluation

2 Drop all labels except **`service_name`**

`by (service_name)`

```
http_request_int_total{service_name="db", status_code=503} 8
http_request_int_total{service_name="api", status_code=401} 5
http_request_int_total{service_name="api", status_code=500} 3
```

↓

```
{service_name="db"} 8
{service_name="api"} 5
{service_name="api"} 3
```

3 Aggregate the metrics

`sum`

```
{service_name="db"} 8
{service_name="api"} 5
{service_name="api"} 3
```

→

```
{service_name="db"} 8
{service_name="api"} 8
```

© Copyright National University of Singapore

Metric Aggregation - Longitudinal Sectional

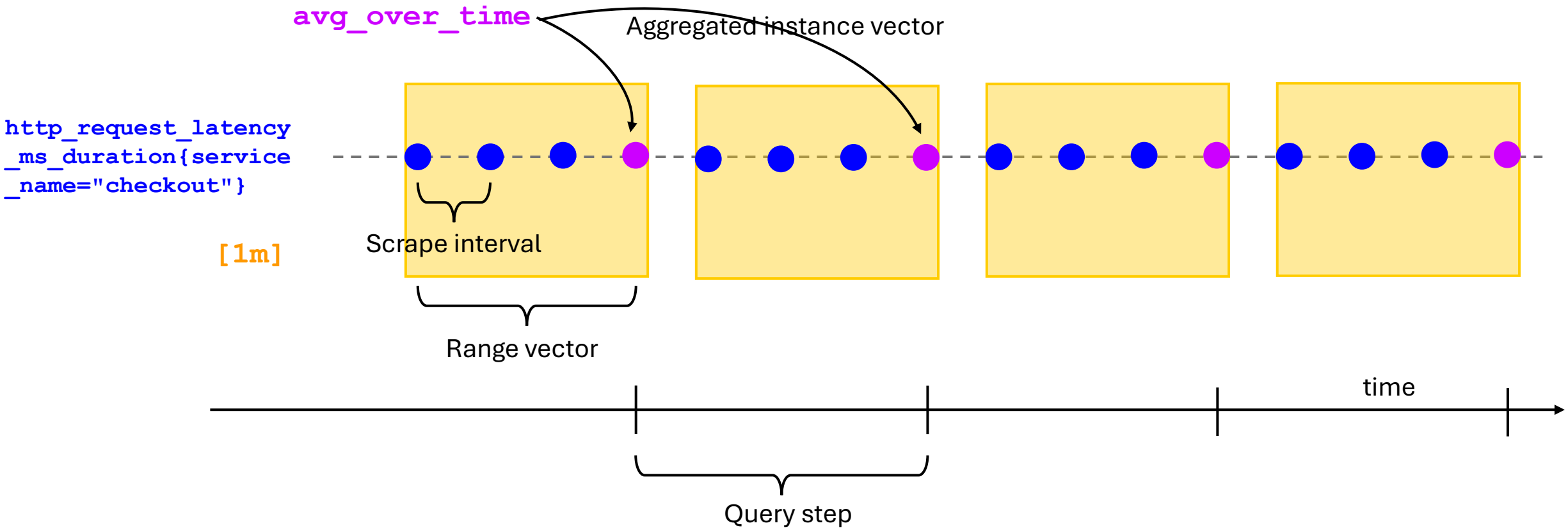
- Aggregate multiple values from a single time series over a defined time range into a single data point
  - Operates on a range vector
- Common aggregation function
  - Rate - **`rate`**, **`irate`**, **`increase`**
  - Function names ending with **`_over_time`**
    - Statistical aggregation - **`avg_over_time`**, **`max/min_over_time`**, **`stddev_over_time`**
    - Counting - **`count_over_time`**, **`changes`**
    - Quantile calculation - **`quantile_over_time`**

© Copyright National University of Singapore

Query Evaluation

Average latency of the checkout service over 1 minute

```
avg_over_time (http_request_latency_ms_duration{service_name="checkout"}[1m])
```



Assume scrape interval is 15 sec

Image adapted from https://iximiuz.com/en/posts/prometheus-functions-agg-over-time/

© Copyright National University of Singapore

`rate`, `irate` and `increase`

- Used to analyze counter metrics
  - Eg. `http_request_total`
  - Handles counter reset viz. counter is restarted

	Output	Method	Use Case
`rate(metric[t])`	Per-second rate	Use the first and the last data point to calculate the average rate over the duration (`t`)	Trend analysis - smooth line for identifying long term trends
`irate(metric[t])`	Instant per-second rate	Use only the last two data point within the duration (`t`) to calculate the instantaneous rate of change	Spikes - show recent changes in sudden spikes or drop in activities; for spotting recent issues
`increase(metric[t])`	Total count	Calculate the absolute increase in the counter's value over the duration (`t`)	Total count - for total number of events occurred

© Copyright National University of Singapore

Example - `rate`, `irate` and `increase`

`http_request_total[2m]` = 4, 5, 7, 9, 12, 14, 17, 20

`rate(http_request_total[2m])`

$$\frac{(20 - 4)}{120} = 0.1333$$

`irate(http_request_total[2m])`

$$\frac{(20 - 17)}{15} = 0.2$$

`increase(http_request_total[2m])`

$$(20 - 4) = 16$$

© Copyright National University of Singapore

Common Metrics

- Average response time over the last hour

```
sum(rate(http_request_duration_seconds_sum[5m])) /
  sum(rate(http_request_duration_seconds_count[5m]))
```

- 95th percentile of service response time over the last 30 minutes must be less than 1000ms

```
quantile_over_time(0.95,
  http_request_duration_ms[30m]) < 1000
```

- Changes in the number of uncheckout carts

```
delta(cart_uncheckout[30m])
```

- Predict the available memory over the next 5 hours based on the usage patterns of the last hour

```
predict_linear(node_memory_MemAvailable_bytes{env="prod"}[1h],
  5 * 60 * 60)
```

© Copyright National University of Singapore

Common Metrics

- Errors rate over the last 15 minutes

```
rate(http_request_total{status=~"[45].."}[15m])
```

- Request rate by endpoint over the last 15 minutes

```
sum by (method, path) (rate(http_request_total)[15m])
```

- Error rate over time

```
(sum(rate(http_request_total{status=~"[45].."}[15m])) /
  sum(rate(http_request_total[15m]))) * 100
```

© Copyright National University of Singapore

Quantiles

- OTel SDK does not produce quantiles, but can be calculated from histograms
  - Prometheus summaries
- To calculate quantiles from histograms
  - Calculate the rate of change for each bucket (`le=`) over a time window eg. 5m

```
rate(http_request_duration_ms_bucket[5m])
```

  - Group and aggregate the metrics if required
    - Eg. group all `method=POST`, `uri=/checkout` by their bucket (`le`)

```
sum by(method, uri, le)(rate(http_request_duration_ms_bucket[5m]))
```

  - Use the `histogram_quantile` function to get the required quantile between 0 and 1

```
histogram_quantile(.99,
  sum by(method, uri, le)(rate(http_request_duration_ms_bucket[5m])))
```

© Copyright National University of Singapore

Workshop 3
Task 3

© Copyright National University of Singapore

Service Graphs

© Copyright National University of Singapore

Service Graphs

- Service graphs are visual representations of service dependencies and interactions in a micro services mesh
  - Derived from traces stored in Tempo
- Shows
  - Topology - how services communicates with each other
  - Request flow - direction and volume of request between services
  - Performance metrics - RED metrics for each service interaction
  - Health - status of the heath status of services and connections
- Use cases
  - High level view of the system's architecture
  - Performance bottleneck
  - Error propagating through the system
  - Perform what-if impact analysis when services fails

© Copyright National University of Singapore

Example - Service Graphs

Outline | tempo | Split | Add | Last 1 hour | Run query

Name	Rate		Error Rate		Duration (p90)	Links
POST:/api/book	0.01	0.014	0.00	0.004	9.23 s	Tempo
GET /api/books	0.01	0.012	0.00	0.001	10.5 s	Tempo
GET /api/search	0.01	0.012	0.00	0.001	10.3 s	Tempo
POST /api/book	0.01	0.010	0.00	0.001	9.65 s	Tempo
GET /api/book/:isbn13	0.01	0.009	0.00	0.000	10.2 s	Tempo

Node graph

Layered | Force | Grid

© Copyright National University of Singapore

Enabling Service Graphs - 1

- Enable remote write option when starting Prometheus

```
--web.enable-remote-write-receiver
```

  - Tempo write service graphs metrics to Prometheus for house keeping
  - Verify the setup after enabling service graph. Check if the following metrics exist in Prometheus
    - `traces_service_graph_request_total`
    - `traces_service_graph_request_server_seconds`

- Enable Tempo metrics generator and service graphs

```
overrides:
  defaults:
    metrics_generator:
      processors:
      - local-blocks
      - service-graphs
      - span-metrics
      generate_native_histograms: both

metrics_generator:
  storage:
    remote_write:
    - url: http://prometheus:9090/api/v1/write
```

© Copyright National University of Singapore

Enabling Service Graphs - 2

Additional settings

Additional settings are optional settings that can be configured for more control over your data source.

Advanced HTTP settings

Allowed cookies	New cookie (hit enter to add) Add
Timeout	Timeout in seconds

Service graph

Select a Prometheus data source that contains the service graph data. Learn more about the service graph

Data source	prometheus	Clear
Histogram type	Classic	

Node graph

Show or hide the node graph visualization. Learn more about the node graph

Enable node graph	

Tempo search

Add Prometheus to Tempo data source to enable service graphs

© Copyright National University of Singapore

Workshop 3
Task 4

© Copyright National University of Singapore

Span Metrics

© Copyright National University of Singapore

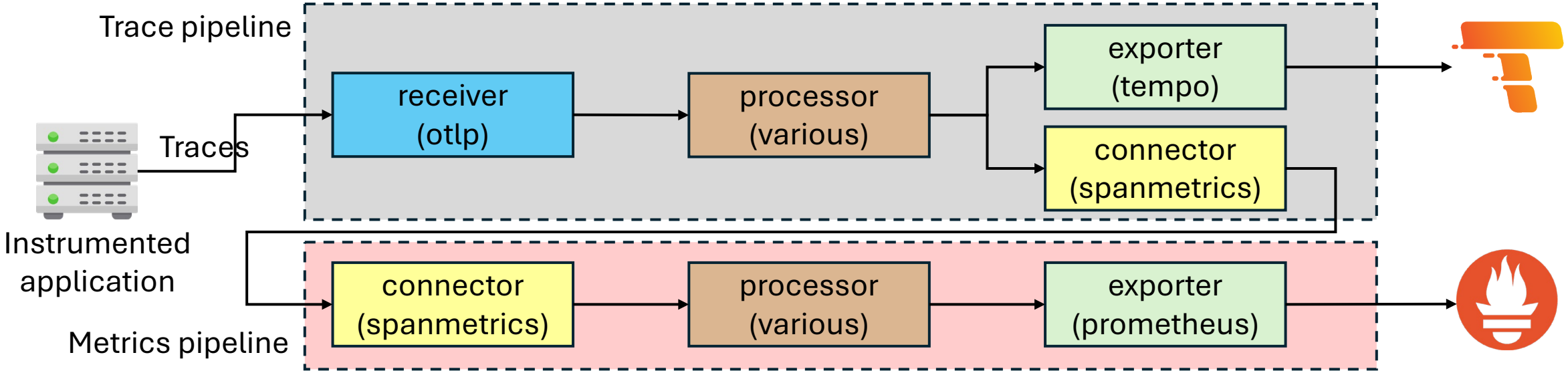
Span Metrics

- Automatically generates/derive RED metrics from trace spans by the OTel collector
  - Rate - call count
  - Error - number of failed request based on span status
  - Duration - duration of the trace
- Reduces instrumentation code

© Copyright National University of Singapore

Span Metrics Architecture

- Implemented as a OTel connector
  - Join two or more pipelines, act as an exporter to one and a receiver to another
- Span metrics connector joins the trace pipeline to the metrics pipeline
  - Generated metrics from ingested traces and export them to the metrics pipeline



© Copyright National University of Singapore

Generated Metrics by Span Metrics

- Request rate metrics
  - **`calls_total`** (counter) - total number of spans processed
    - Labels include service_name, span_name
  - Use to measure throughput and traffic patterns
- **`_duration_milliseconds`** (histogram) - latency distribution of spans
  - Use in quantiles calculation
- **`_duration_milliseconds_sum`** (sum) - total duration of all spans
  - Use for calculating the average latency

© Copyright National University of Singapore

Examples - Span Metrics

```
traces_span_metrics_calls_total{service_name="checkout-
service",span_kind="SPAN_KIND_SERVER",http_method="POST",http_status_code="200"}
15234

traces_span_metrics_duration_milliseconds_bucket{service_name="checkout-
service",span_kind="SPAN_KIND_SERVER",http_method="POST",http_status_code=
"200",le="5"} 1245
traces_span_metrics_duration_milliseconds_bucket{service_name="checkout-
service",span_kind="SPAN_KIND_SERVER",http_method="POST",http_status_code=
"200",le="10"} 3456

traces_span_metrics_duration_milliseconds_sum{service_name="checkout-
service",span_kind="SPAN_KIND_SERVER",http_method="POST",http_status_code="20
0"} 456789.23
traces_span_metrics_duration_milliseconds_count{service_name="checkout-
service",span_kind="SPAN_KIND_SERVER",http_method="POST",http_status_code="20
0"} 15234
```

© Copyright National University of Singapore

Configuring Span Metrics

```
connectors:
  spanmetrics:
    histogram:
      explicit:
        buckets: [ 50ms, 300ms, 500ms, 800ms, 1s, 1500ms, 2s, 3s, 5s, 10s ]

    exemplars:
      enabled: true

    dimensions:
    - name: http.method
    - name: http.status_code
      default: 400

    aggregation_temporality: "AGGREGATION_TEMPORALITY_CUMULATIVE"
```

Add a connector spanmetrics connector to OTel configuration

Explicitly define the histogram buckets, **`le`** label

Enable exemplars to correlate metrics back to traces

Define what labels from the spans are to be included in the metrics. Default labels include **`service.name`**, **`span.name`**, **`status.code`**

Specifies how the metrics are aggregated

© Copyright National University of Singapore

Configuring Histogram Buckets

- Histogram buckets can be configured as
  - Explicit - predefined fixed bucket boundaries
    - Each bucket count values less than or equals to its boundaries (**`le`**)
    - Eg. 50ms, 300ms, 500ms, 800ms, 1s, 1500ms, 2s, 3s, 5s, 10s
  - Exponential - dynamically create bucket boundaries in the power of 2
    - Provides better precision for based on the actual observed latencies
    - Default number of bucket is 160 bucket, can be capped with **`max_size`**
  - Explicit and exponential buckets are mutually exclusive
    - See https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/connector/spanmetricsconnector#configurations

© Copyright National University of Singapore

Histogram Buckets Best Practices

- Span metrics histogram buckets determines the metrics latency resolution
  - Too few buckets, lose granularity
    - Analysis will not be accurate, impact SLO calculation e.g. 200ms and 800ms in the same bucket if the bucket starts with 1000ms
    - Cannot identify issues and trends
  - Too many buckets, increase cardinality
    - Increase storage, slower metric ingestion, diminishing returns
    - Impact Prometheus queries
- Tailor buckets to the service's latency profile
  - Fast services - e.g. APIs, fast response, smaller bucket sizes in milliseconds
  - Slower services - e.g. checkouts, batch jobs, slower services, larger buckets in seconds

© Copyright National University of Singapore

Configuring Span Metrics

```yaml
receivers:
  otlp:
    protocols:
      ...

processors:
  batch:

connectors:
  spanmetrics:
    ...

exporters:
  prometheus:
    endpoint: 0.0.0.0:9090
    send_timestamps: true
    translation_strategy: UnderscoreEscapingWithSuffixes
    enable_open_metrics: true
```

```yaml
service:
  pipelines:
    traces:
      receivers: [ otlp ]
      processors: [ batch ]
      exporters: [ otlp/tempo, spanmetrics ]
    metrics:
      receivers: [ otlp, spanmetrics]
      processors: [ batch ]
      exporters: [ prometheus ]
```

Generated metrics are fed to a metrics receiver pipeline

Enable OpenMetrics format to capture exemplars

© Copyright National University of Singapore

RED Metrics from Span Metrics

- Request rate

```
rate(traces_span_metrics_calls_total[5m])
```

- Error rate

```
(rate(traces_span_metrics_calls_total{http_status_code =~ "[45]..."}[5m])
   / rate(traces_span_metrics_calls_total[5m])) * 100
```

- Duration

```
histogram_quantile(.95,
    rate(traces_span_metrics_duration_milliseconds_bucket[5m]))
```

© Copyright National University of Singapore

Span Metrics Single Write Principle

- Every span is written exactly once to its corresponding metric time series
  - Ensures consistency and no double counting
- Avoid the following
  - Include high dimension labels from spans to the generated metrics like **`sku`**, **`user_id`**
  - Processing spans multiple times by difference span metrics
    - Only one spanmetrics connector should be added to a trace pipeline
  - See https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/connector/spanmetricsconnector#known-limitation-the-single-writer-principle

© Copyright National University of Singapore

Example of Duplicate Span Metrices

```
connectors:
  spanmetrics/explicit
    histogram:
      explicit:
        ...

  spanmetrics/exponential
    histogram:
      exponential:
        ...

service:
  pipelines:
    trace:
      receivers: [ otlp ]
      exporters: [ otlp/tempo, spanmetrics/explicit, spanmetrics/exponential ]
```

Spans from a single receive are processed by 2 separate spanmetrics connector.

Results in duplicate time series e.g. **`trace_span_metrics_total_calls`** is created twice; Prometheus will report an error

© Copyright National University of Singapore

Exemplars

© Copyright National University of Singapore

What are Exemplars?

- Feature that provide links from metrics back to the specific example traces that contributed to those metric values
  - Additional field in the metric that includes the **`traceId`** of the span that generated the metrics
  - Bridge the gap between what (metrics) and why (traces)
  - Eg. Find specific traces from metrics e.g. slow traces observed from metrics
- Exemplars can be associated with any metrics
  - Histograms and summaries to provide trace examples of the distributed values
    - Eg. Linking the **`le=1000`** latency metrics to traces that generated it
  - Not meaningful for counters and gauges

© Copyright National University of Singapore

Exemplars from Span Metrics

- Span metrics can correlated derived metrics back to the original traces through exemplars
- Need to be enabled
  - spanmetrics connector
  - Enable exemplar storage in Prometheus

© Copyright National University of Singapore

Enabling Exemplars from Traces

- Enable exemplars in OTel collector
  - Metrics must also be exported to Prometheus in Open Metrics format

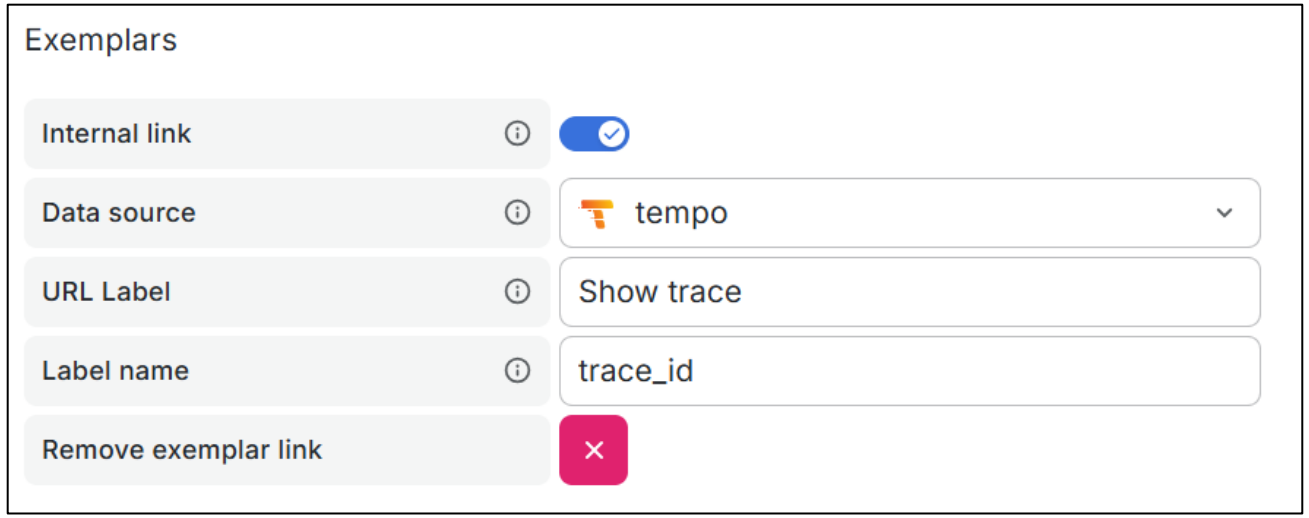
```
connectors:
  spanmetrics:
    exemplars:
      enabled: true

exporters:
  prometheus:
    enable_open_metrics: true
```

- Start Prometheus with exemplar storage flag

```
--enable-feature=exemplar-storage
```

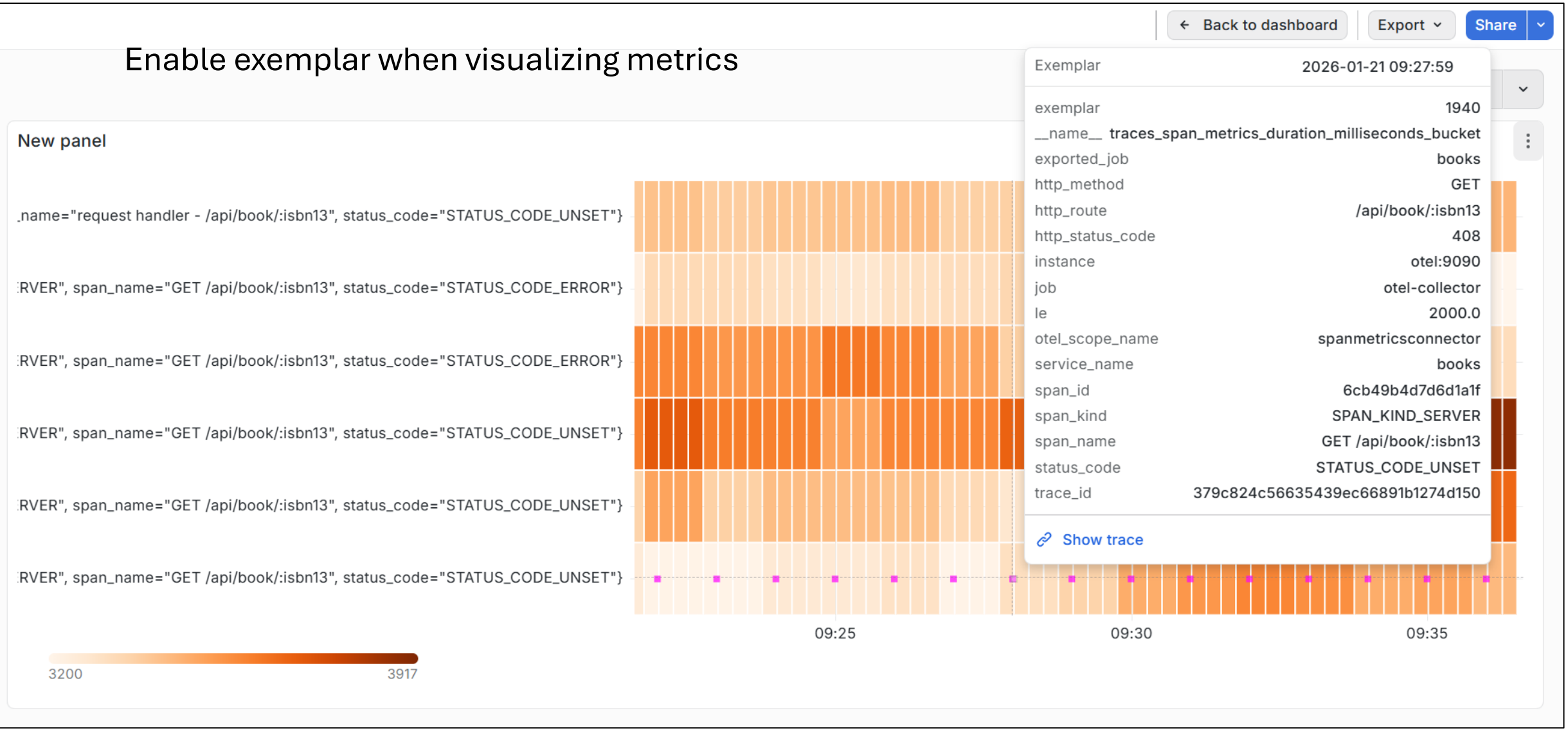
- Enable Exemplar in Grafana Prometheus data source



© Copyright National University of Singapore

Example - Exemplars

Enable exemplar when visualizing metrics



© Copyright National University of Singapore

Workshop 3
Task 5

© Copyright National University of Singapore

End of Module

© Copyright National University of Singapore